

Digital Logic Design + Computer Architecture

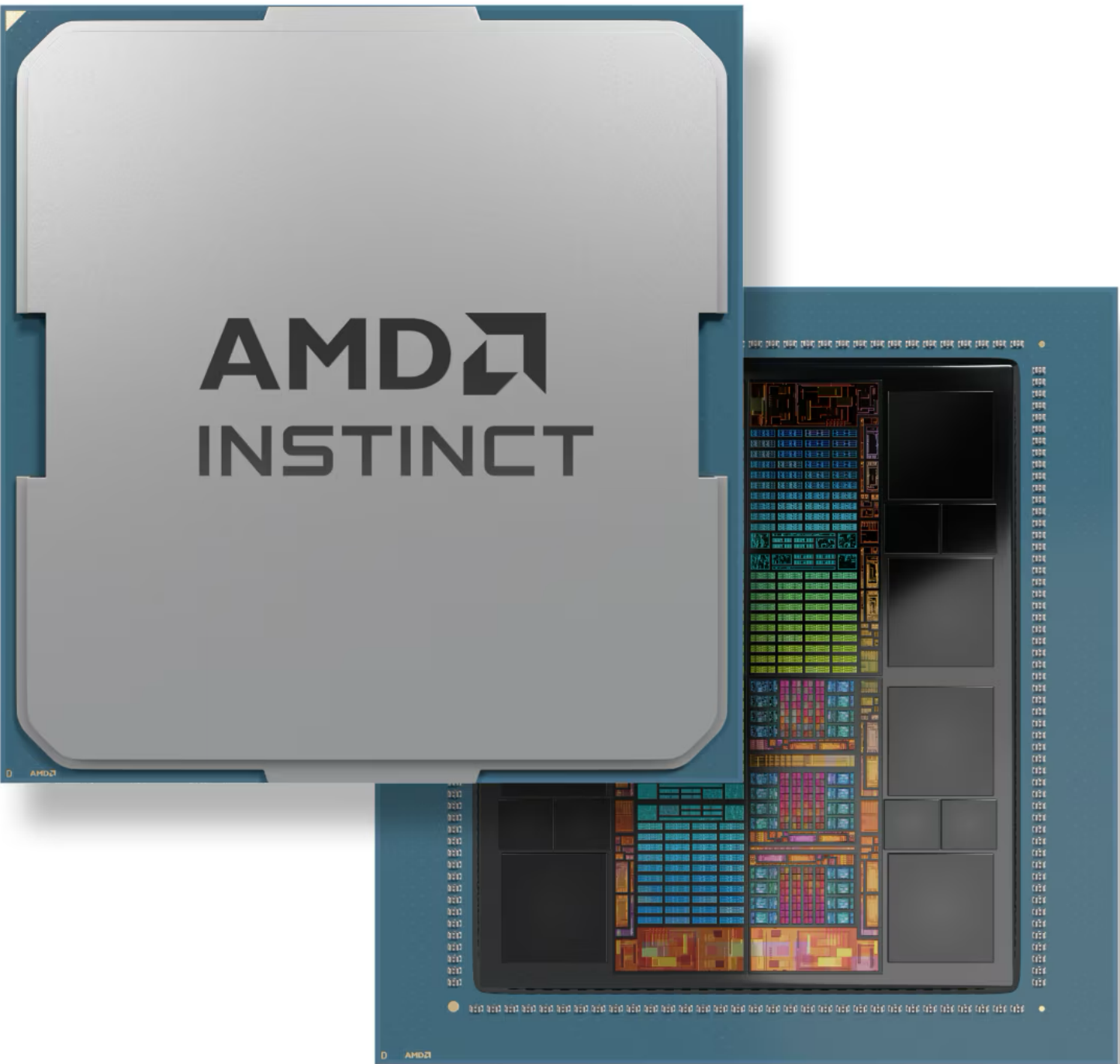
Sayandeep Saha

Assistant Professor
Department of Computer
Science and Engineering
Indian Institute of Technology
Bombay



Combinational Circuits

Do You Want to Design Some Day?



Design with Gates

- **Logic gates:** perform logical operations on input signals
- **Positive (negative) logic polarity:** constant 1 (0) denotes a high voltage and constant 0 a low (high) voltage
- **Combinational circuits:** No memorization
- **Synchronous sequential circuits:** have memory; driven by a clock that produces a train of equally spaced pulses
- **Propagation delay:** time to propagate a signal through a gate
- **Asynchronous circuits:** are almost free-running and do not depend on a clock; controlled by initiation and completion signals

Exclusive-OR (XOR) and XNOR

Exclusive-OR: modulo-2 addition, i.e., $A \oplus B = 1$ if either A or B is 1, but not both.

Commutativity: $A \oplus B = B \oplus A$

Associativity: $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$

Distributivity: $(AB) \oplus (AC) = A(B \oplus C)$

If $A \oplus B = C$, then

$$A \oplus C = B$$

$$B \oplus C = A$$

$$A \oplus B \oplus C = 0$$

Exclusive-OR of an even (odd) number of elements, whose value is 1, is 0 (1)

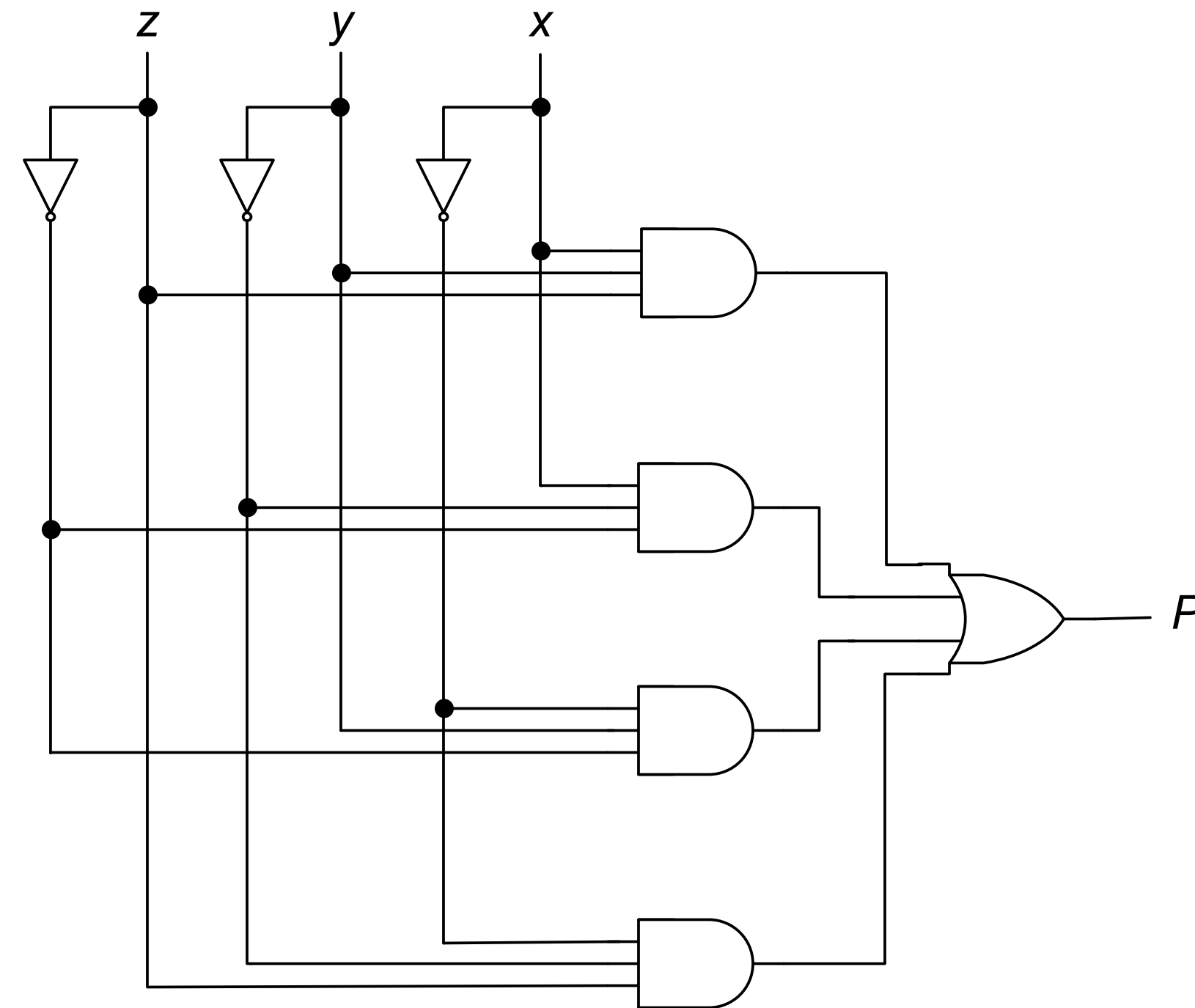
XNOR: Complement of XOR

Combinational Circuits: Parity-bit Generator

Parity-bit generator: produces output value 1 if and only if an odd number of its inputs have value 1

xy		z			
		00	01	11	10
z	0	0	1	0	1
	1	1	0	1	0

(a) Map.



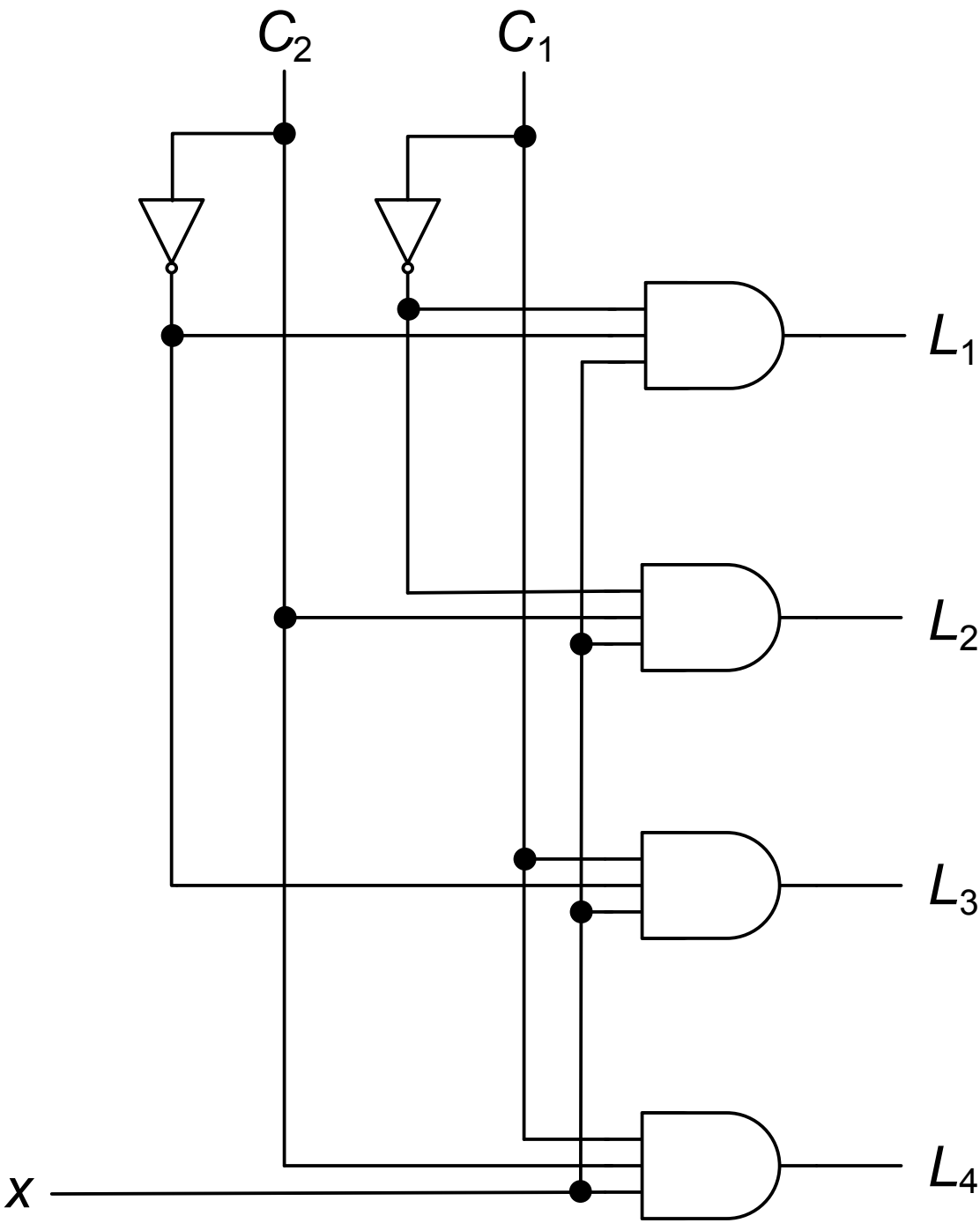
(b) Implementation.

$$P = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z$$

Combinational Circuits: Serial to Parallel

Serial-to-parallel converter: distributes a sequence of binary digits on a serial input to a set of different outputs, as specified by external control signals

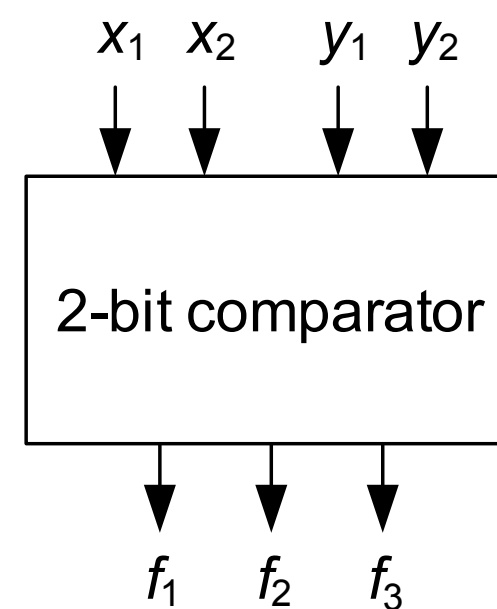
<i>Control</i>		<i>Output lines</i>				<i>Logic equations</i>
C_1	C_2	L_1	L_2	L_3	L_4	
0	0	x	0	0	0	$L_1 = xC_1' C_2'$
0	1	0	x	0	0	$L_2 = xC_1' C_2$
1	0	0	0	x	0	$L_3 = xC_1 C_2'$
1	1	0	0	0	x	$L_4 = xC_1 C_2$



Combinational Circuits: Comparators

n -bit comparator: compares the magnitude of two numbers X and Y , and has three outputs f_1, f_2 , and f_3

- $f_1 = 1$ iff $X > Y$
- $f_2 = 1$ iff $X = Y$
- $f_3 = 1$ iff $X < Y$



(a) Block diagram.

$$f_1 = ?$$

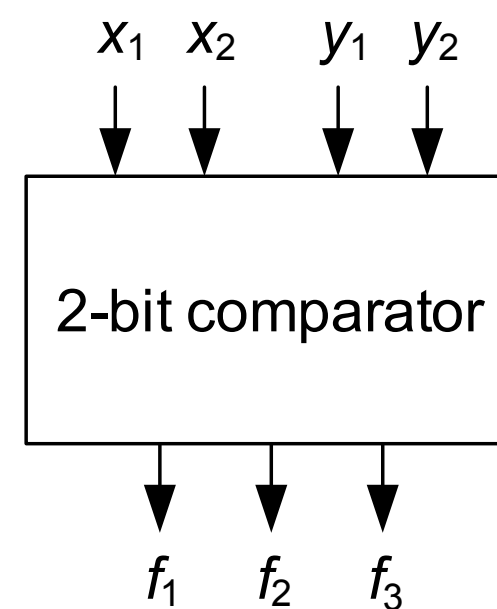
$$f_2 = ?$$

$$f_3 = ?$$

Combinational Circuits: Comparators

n -bit comparator: compares the magnitude of two numbers X and Y , and has three outputs f_1, f_2 , and f_3

- $f_1 = 1$ iff $X > Y$
- $f_2 = 1$ iff $X = Y$
- $f_3 = 1$ iff $X < Y$



(a) Block diagram.

		x_1x_2			
		00	01	11	10
y_1y_2	00	2	1	1	1
	01	3	2	1	1
	11	3	3	2	3
	10	3	3	1	2

(b) Map for f_1, f_2 , and f_3 .

$$f_1 = ?$$

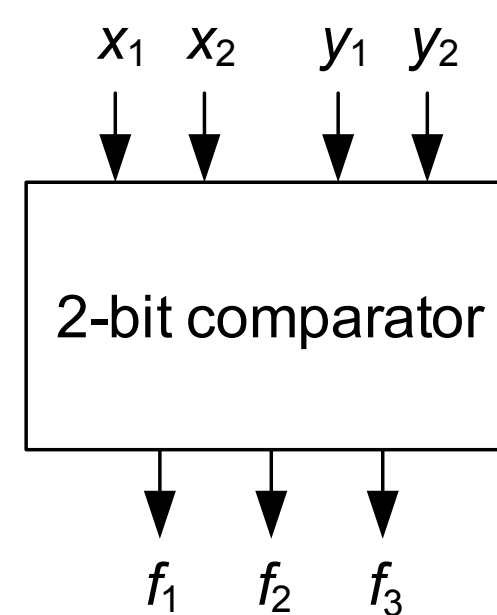
$$f_2 = ?$$

$$f_3 = ?$$

Combinational Circuits: Comparators

n -bit comparator: compares the magnitude of two numbers X and Y , and has three outputs f_1, f_2 , and f_3

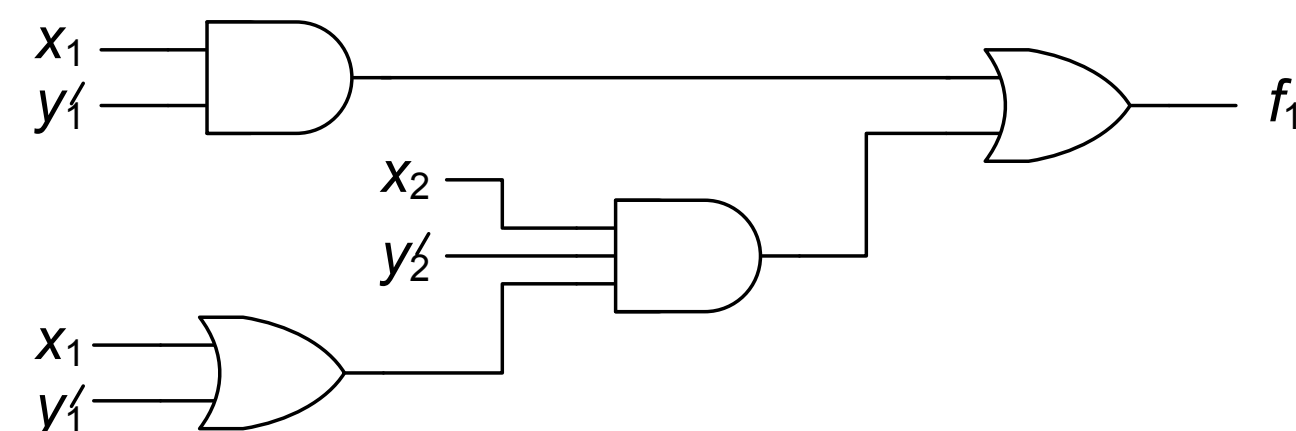
- $f_1 = 1$ iff $X > Y$
- $f_2 = 1$ iff $X = Y$
- $f_3 = 1$ iff $X < Y$



(a) Block diagram.

		x_1x_2			
		00	01	11	10
y_1y_2	00	2	1	1	1
	01	3	2	1	1
	11	3	3	2	3
	10	3	3	1	2

(b) Map for f_1, f_2 , and f_3 .



(c) Circuit for f_1 .

$$f_1 = x_1x_2y_2' + x_2y_1'y_2' + x_1y_1'$$

$$= (x_1 + y_1')x_2y_2' + x_1y_1'$$

$$f_2 = x_1'x_2'y_1'y_2' + x_1'x_2y_1'y_2 + x_1x_2'y_1y_2' + x_1x_2y_1y_2$$

$$= x_1'y_1'(x_2'y_2' + x_2y_2) + x_1y_1(x_2'y_2' + x_2y_2)$$

$$= (x_1'y_1' + x_1y_1)(x_2'y_2' + x_2y_2)$$

$$f_3 = x_2'y_1y_2 + x_1'x_2'y_2 + x_1'y_1$$

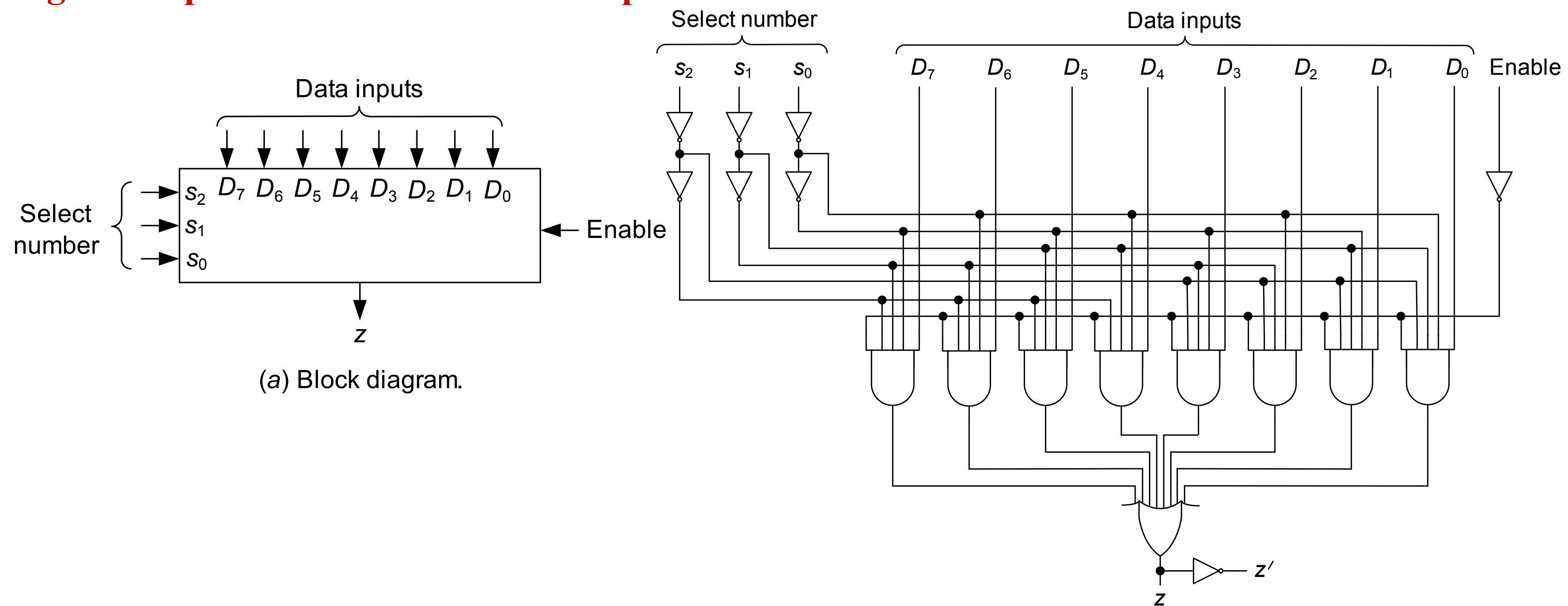
$$= x_2'y_2(y_1 + x_1') + x_1'y_1$$

Combinational Circuits: Multiplexers

Multiplexer: electronic switch that connects one of n inputs to the output

Data selector: application of multiplexer

- n data input lines, $D_0, D_1, \dots, D_{n-1}$
- m select digit inputs $s_0, s_1, \dots, s_{m-1}$
- 1 output
- **Can you design a simple data selectors with 2 input data lines?**

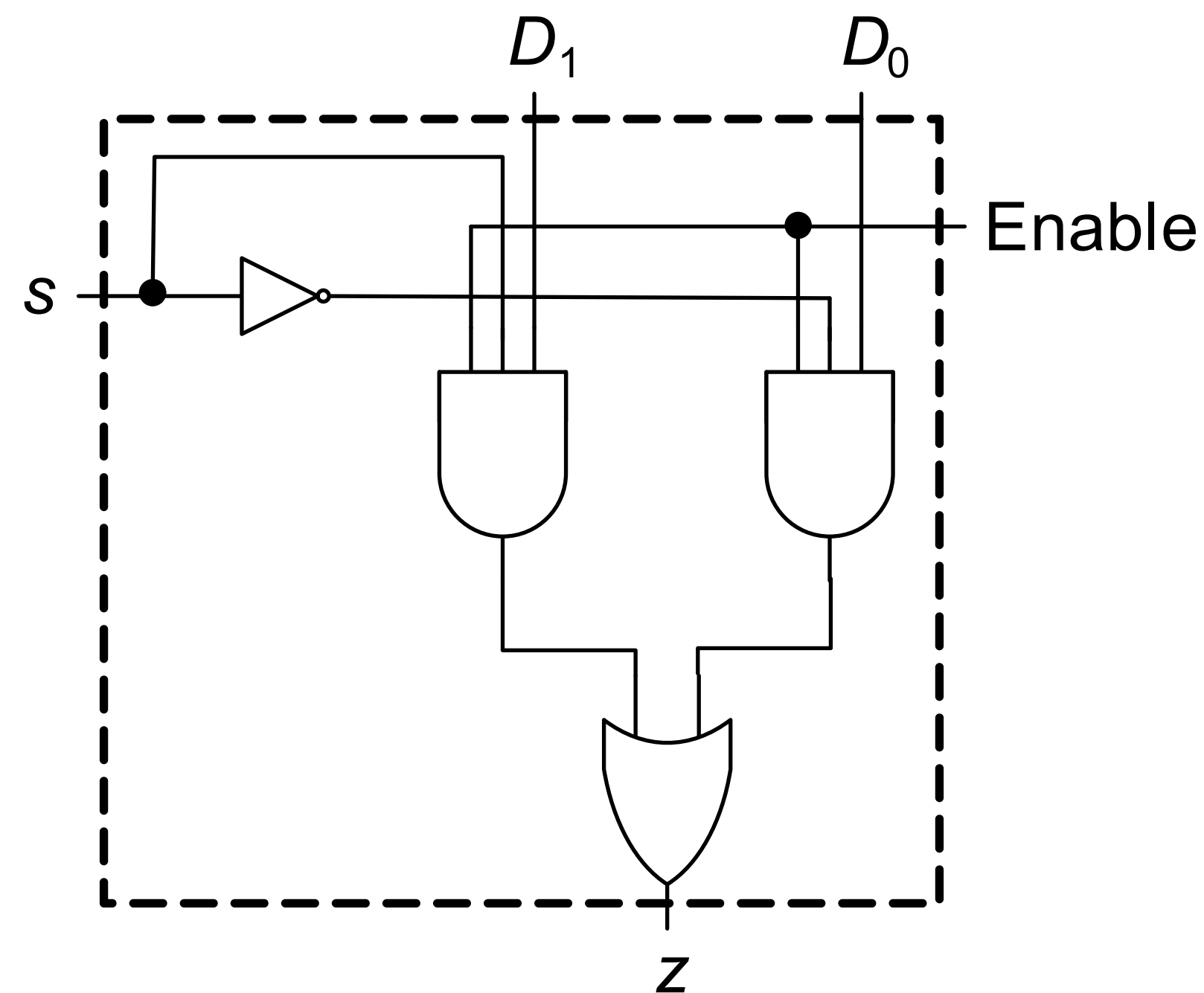


(b) Logic diagram.

Combinational Circuits: Multiplexers

Data selectors: can implement arbitrary switching functions

Example: implementing two-variable functions



$$z = sD_1 + s'D_0$$

If $s = A$, $B = D_0$, and $B' = D_1$, then $z = A \oplus B$.

Implementing Switching Function with Mux

To implement an n -variable function: a data selector with $n-1$ select inputs and 2^{n-1} data inputs

Implementing three-variable functions:

$$z = s_2's_1'D_0 + s_2's_1D_1 + s_2s_1'D_2 + s_2s_1D_3$$

Example: $s_1 = A, s_2 = B, D_0 = C, D_1 = 1, D_2 = 0, D_3 = C'$

$$\begin{aligned} z &= A'B'C + AB' + ABC' \\ &= AC' + B'C \end{aligned}$$

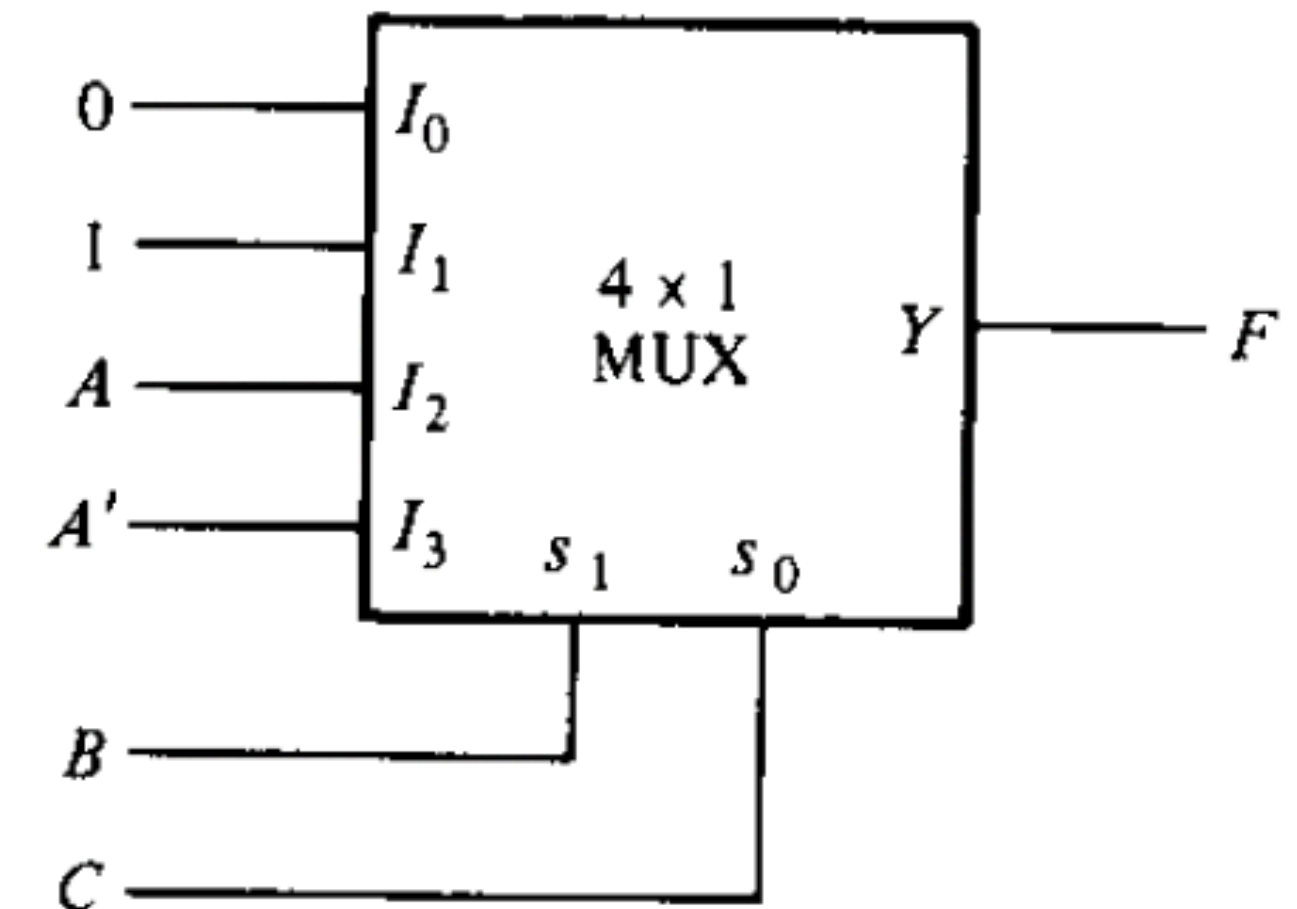
General case: Assign $n-1$ variables to the select inputs and last variable and constants 0 and 1 to the data inputs such that desired function results

Implementing Switching Function with Mux

$$F(A, B, C) = \sum (1, 3, 5, 6)$$

Minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

	I_0	I_1	I_2	I_3
A'	0	①	2	③
A	4	⑤	⑥	7
	0	1	A	A'



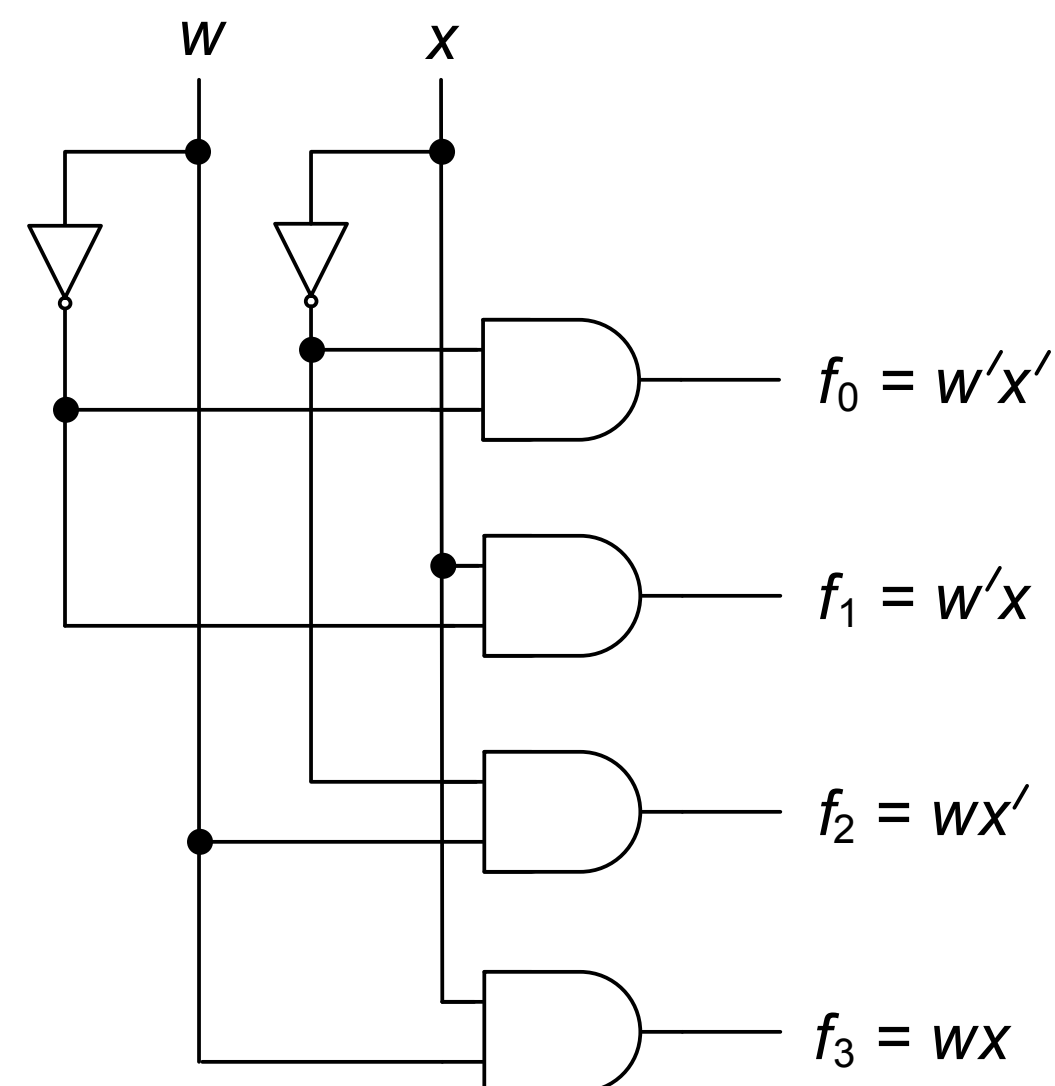
Decoders

Decoders with n inputs and 2^n outputs: for any input combination, only one output is 1

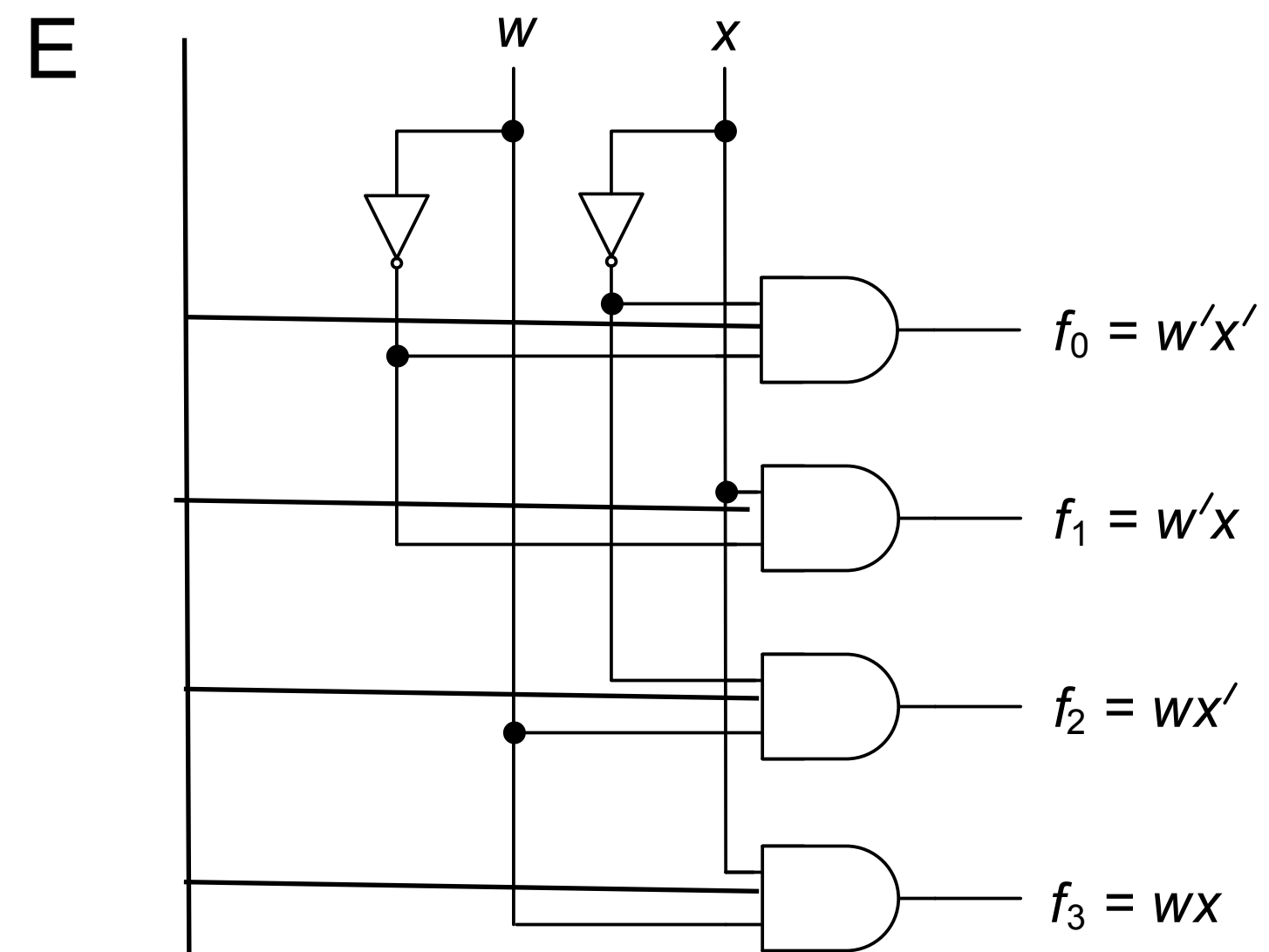
Useful for:

- Routing input data to a specified output line, e.g., in addressing memory
- Basic building blocks for implementing arbitrary switching functions
- Code conversion
- Data distribution

Example: 2-to-4- decoder

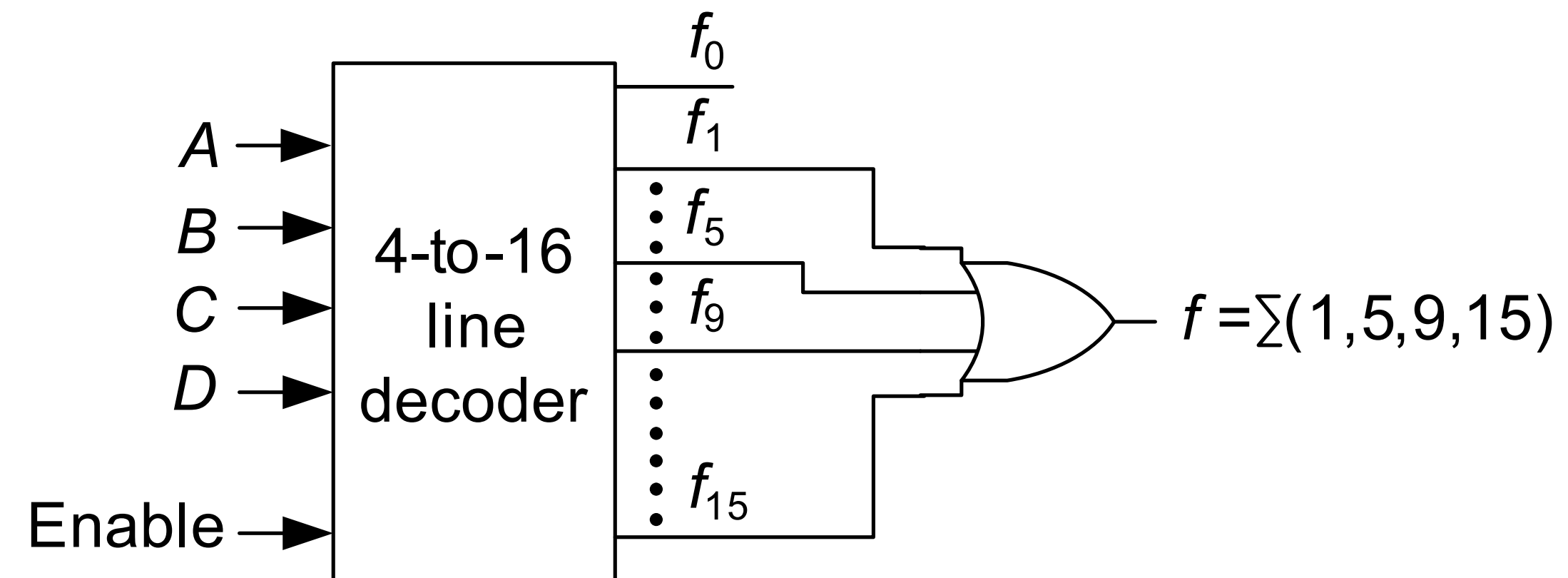


Decoders with Enable



Realizing Arbitrary Functions

Idea: Realize a distinct minterm at each output



Adders: Half Adder

Add two variables and generate the sum and carry

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = x \oplus y$$

$$C = xy$$

Adders: Full Addder

Add two variables and an input carry..

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Try it yourself...!!!

Adders: Full Addder

Add two variables and an input carry..

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = x \oplus y \oplus z$$

$$C = xy + yz + zx$$

Adders: Full Adder with Half Adders

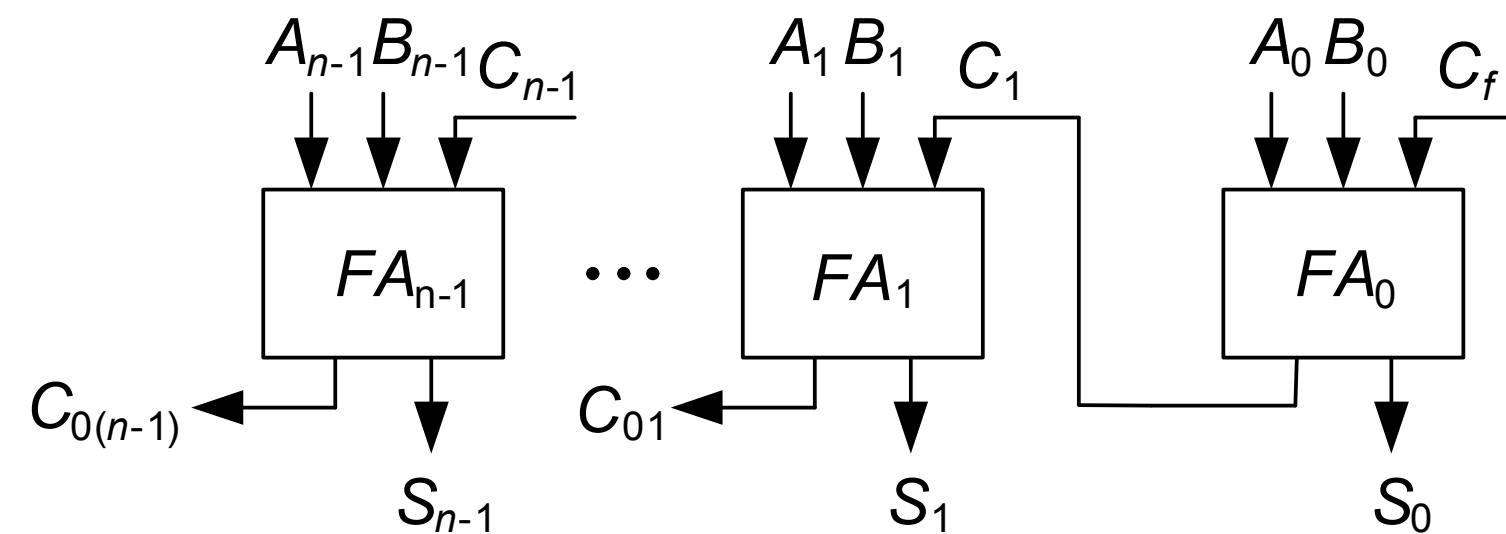
Use two half adder and something else to generate a full addder

Try it yourself...!!!

Ripple Carry Adder

Ripple-carry adder: Stages of full adders

- C_f : forced carry
- $C_{0(n-1)}$: overflow carry



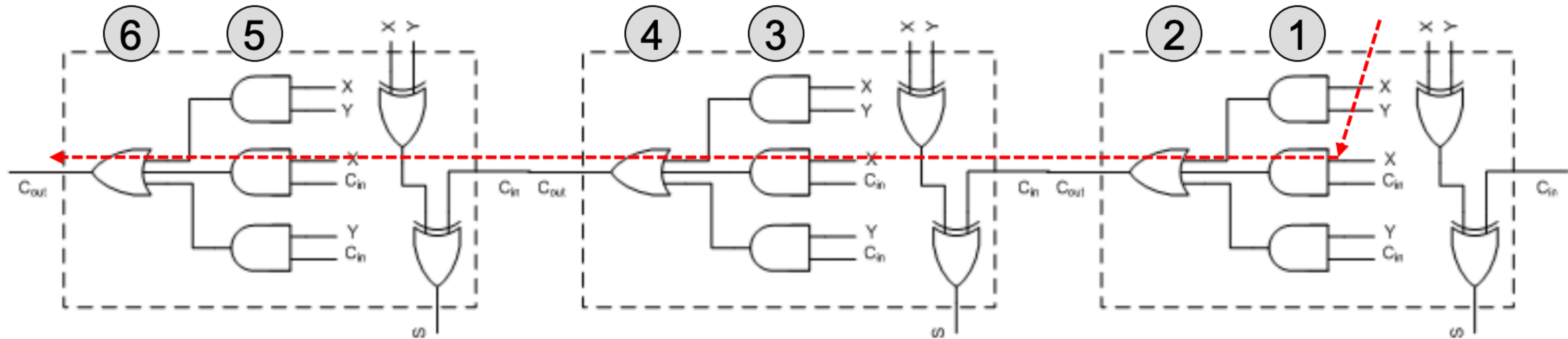
$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{0i} = A_i B_i + B_i C_{0i} + C_{0i} A_i$$

Time required:

- **Carry propagation takes longest time — in the worst case, the carry propagates through all the stages**
- Time per full adder: 2 units (assuming each gate takes one unit of time)
 - Time for carry generation
 - Assumption: two level circuit realisation with 2 input gates
- Time for ripple-carry adder: **$2n$ units**

Ripple Carry Adder



Carry Lookahead Adder

Carry-lookahead adder: several stages simultaneously examined and their carries generated in parallel

- Generate signal $D_i = A_i B_i$
- Propagate signal $T_i = A_i \oplus B_i$
- Thus, $C_{0i} = D_i + T_i C_i$

To generate carries in parallel: convert recursive form to nonrecursive

$$C_{0i} = D_i + T_i C_i$$

$$C_i = C_{0(i-1)}$$

$$\begin{aligned} C_{0i} &= D_i + T_i(D_{i-1} + T_{i-1}C_{i-1}) \\ &= D_i + T_i D_{i-1} + T_i T_{i-1}(D_{i-2} + T_{i-2}C_{i-2}) \\ &= D_i + T_i D_{i-1} + T_i T_{i-1} D_{i-2} + T_i T_{i-1} T_{i-2} C_{i-2} \end{aligned}$$

... ..

$$C_{0i} = D_i + T_i D_{i-1} + T_i T_{i-1} D_{i-2} + \dots + T_i T_{i-1} T_{i-2} \dots T_0 C_f$$

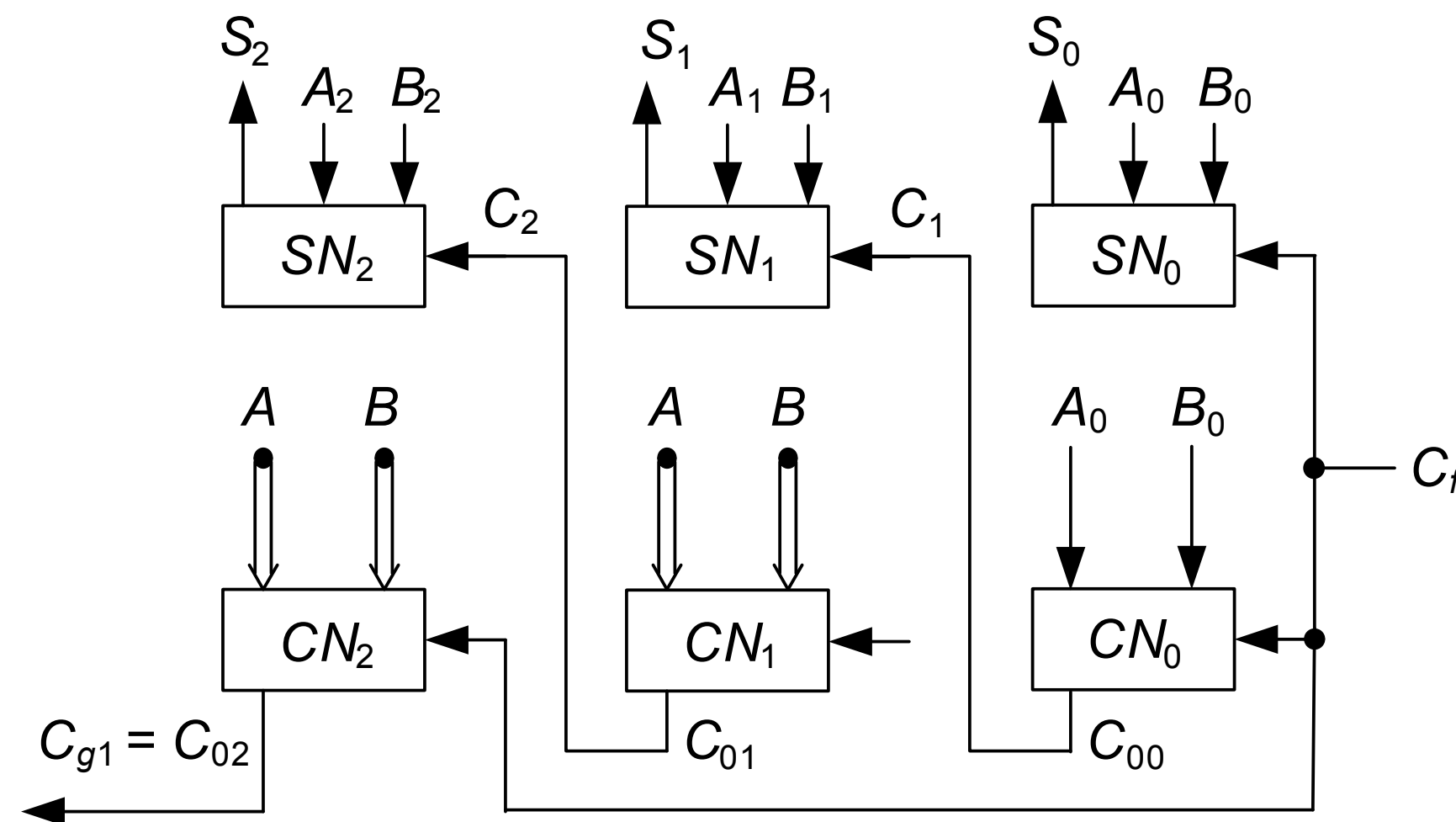
Thus, $C_{0i} = 1$ if it has been generated in the i^{th} stage or originated in a preceding stage and propagated to all subsequent stages

Carry Lookahead Adder

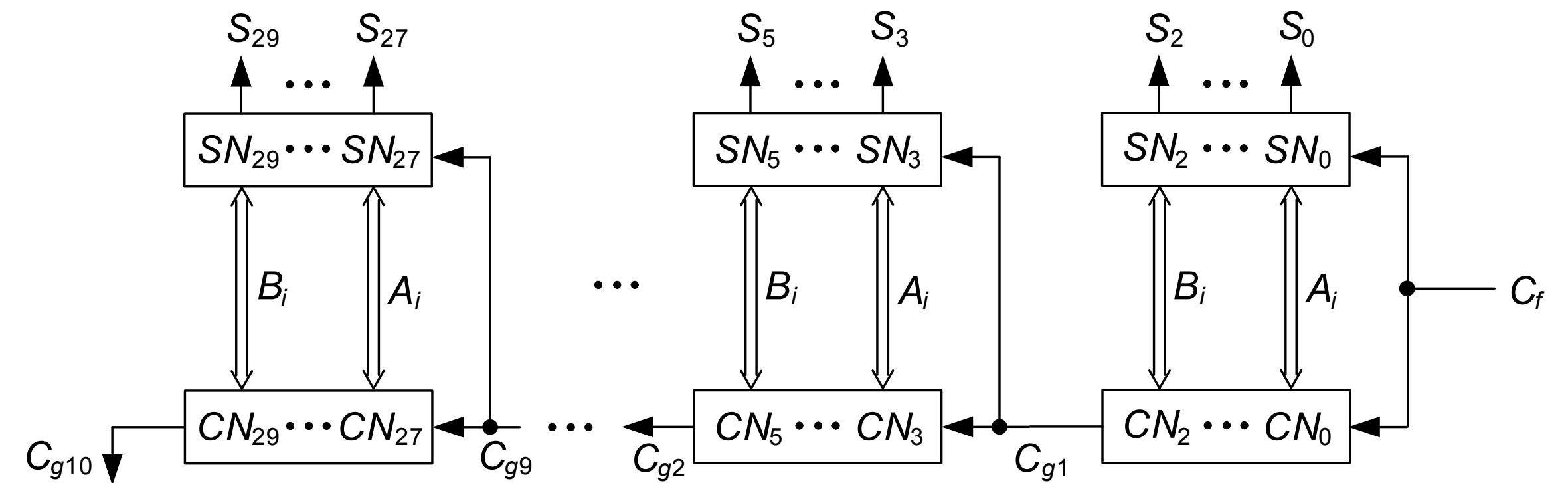
Implementation of lookahead for the complete adder impractical:

- Divide the n stages into groups
- Full carry lookahead within group
- Ripple carry between groups

Example: Three-digit adder group with full carry lookahead



(a) Block diagram of initial three-stage group

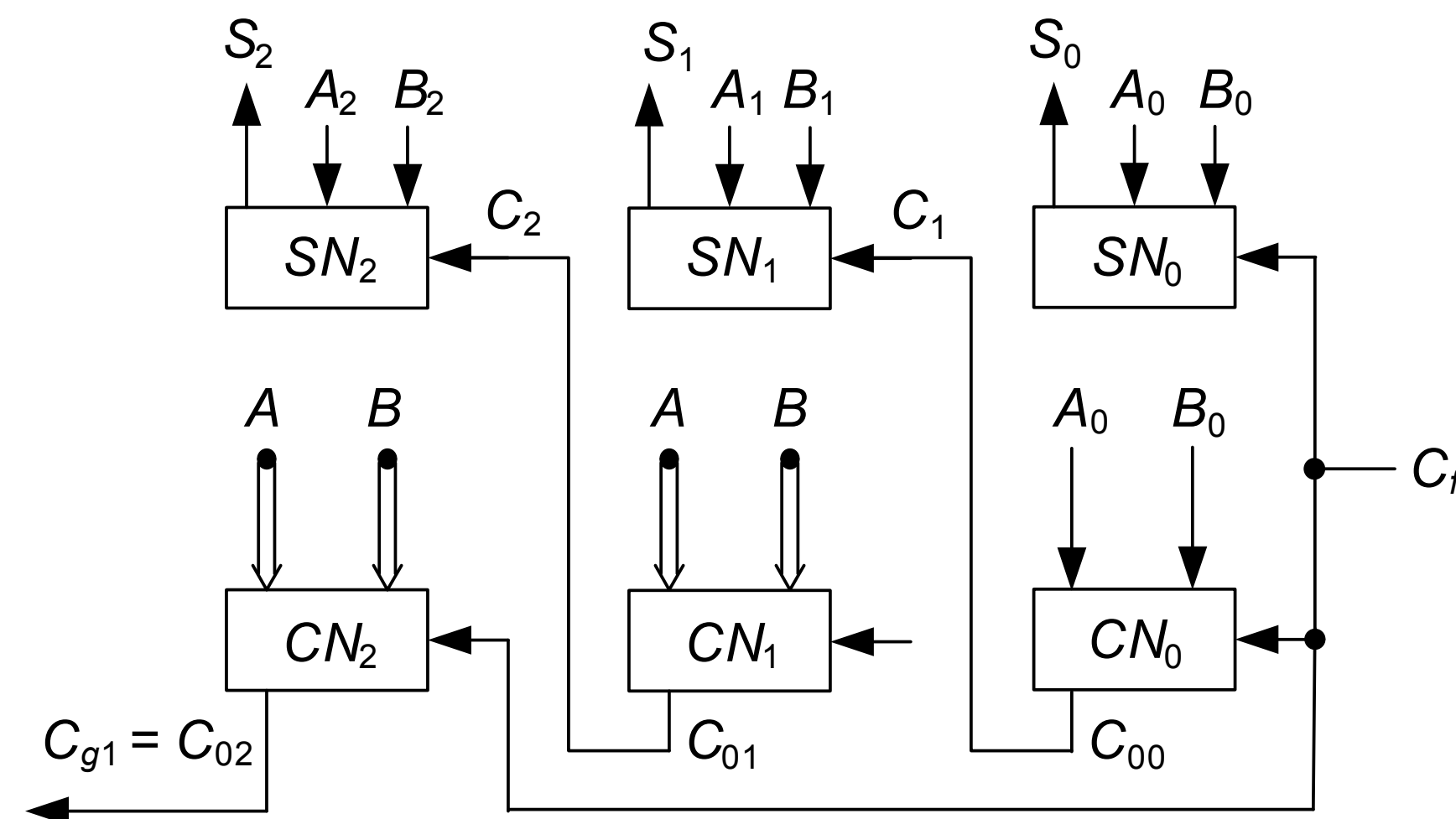


Carry Lookahead Adder

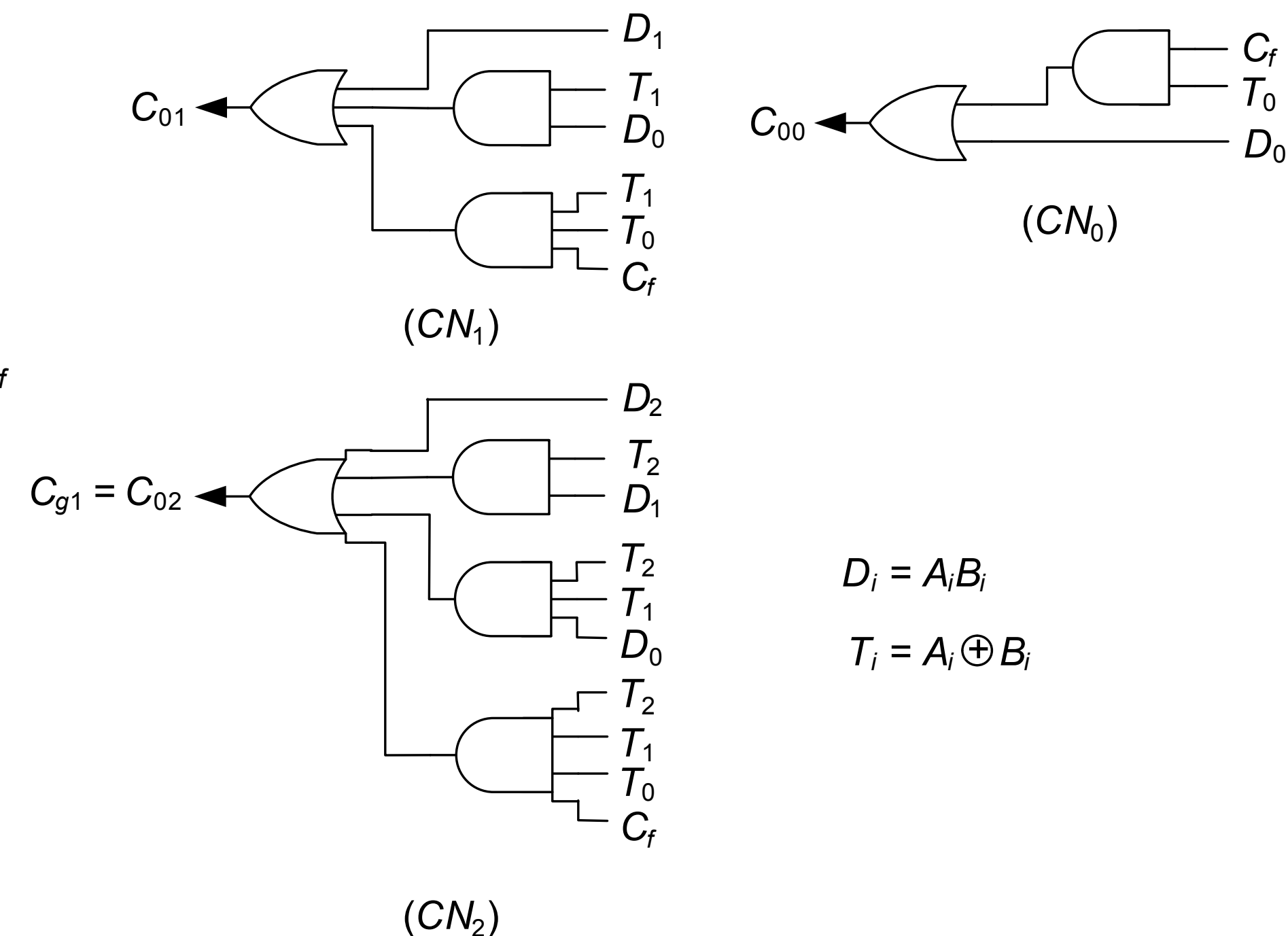
Implementation of lookahead for the complete adder impractical:

- Divide the n stages into groups of 3-bit adders
- Full carry lookahead within group
- Ripple carry between groups

Example: Three-digit adder group with full carry lookahead



(a) Block diagram of initial three-stage group



(b) The carry networks

Time taken:

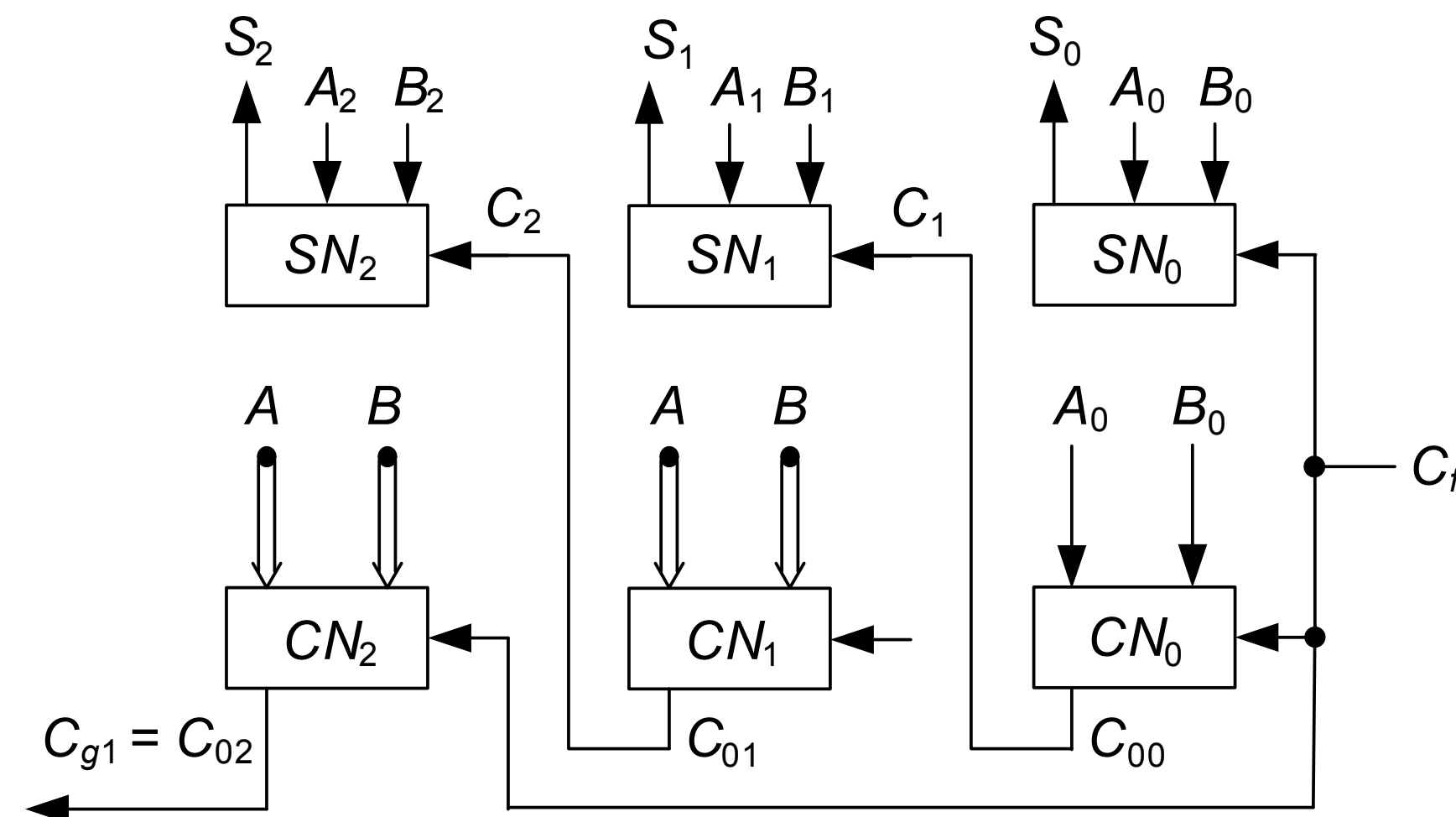
- 4 time units for C_{g1} **why 4?**
- Only 2 time units for C_{g2} and other group carries **why?**

Carry Lookahead Adder

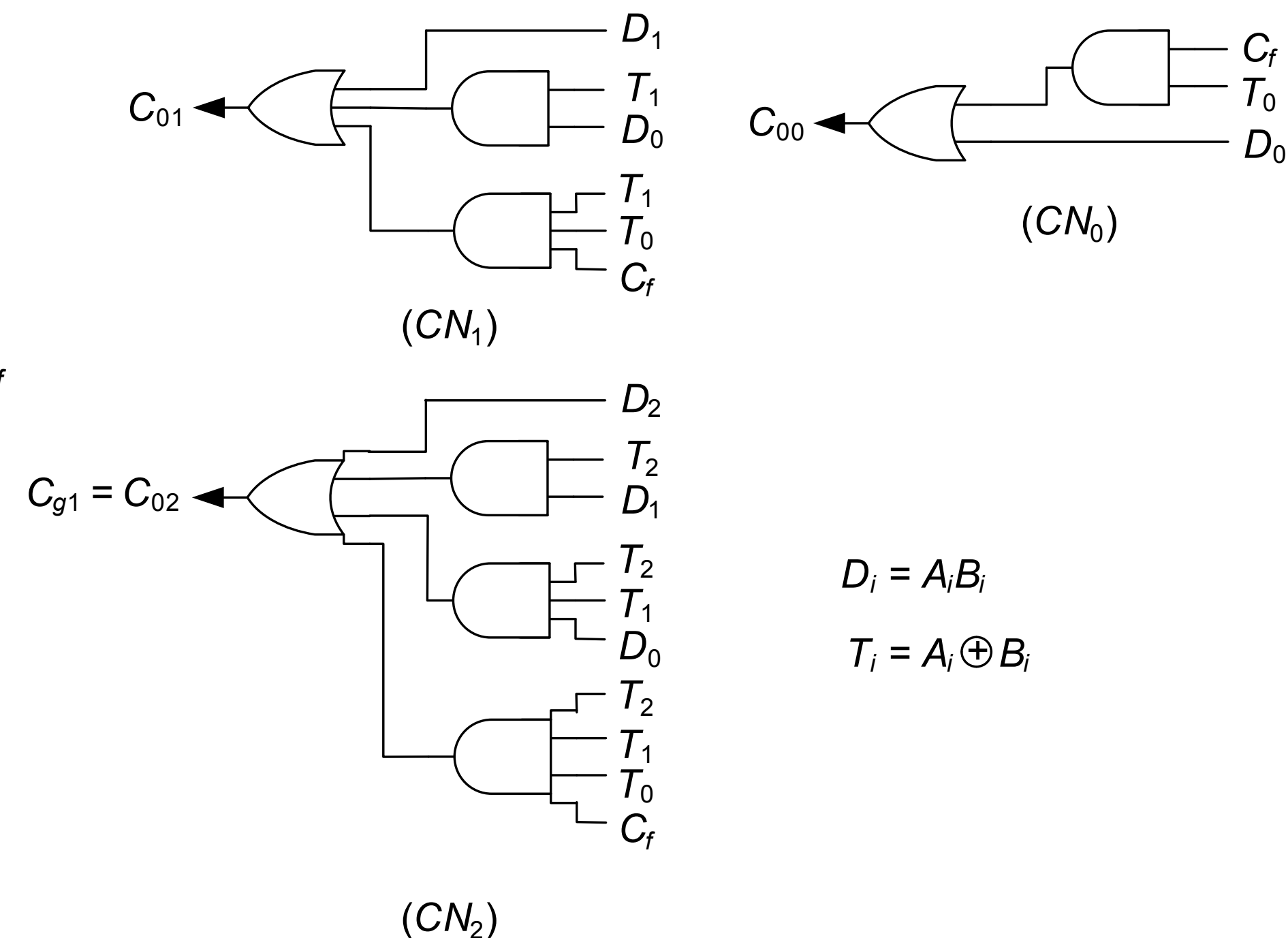
Implementation of lookahead for the complete adder impractical:

- Divide the n stages into groups of 3-bit adders
- Full carry lookahead within group
- Ripple carry between groups

Example: Three-digit adder group with full carry lookahead



(a) Block diagram of initial three-stage group



(b) The carry networks

- XOR - 2 unit delay
- AND - 1 unit delay
- OR - 1 unit delay
- No restriction on the number of inputs

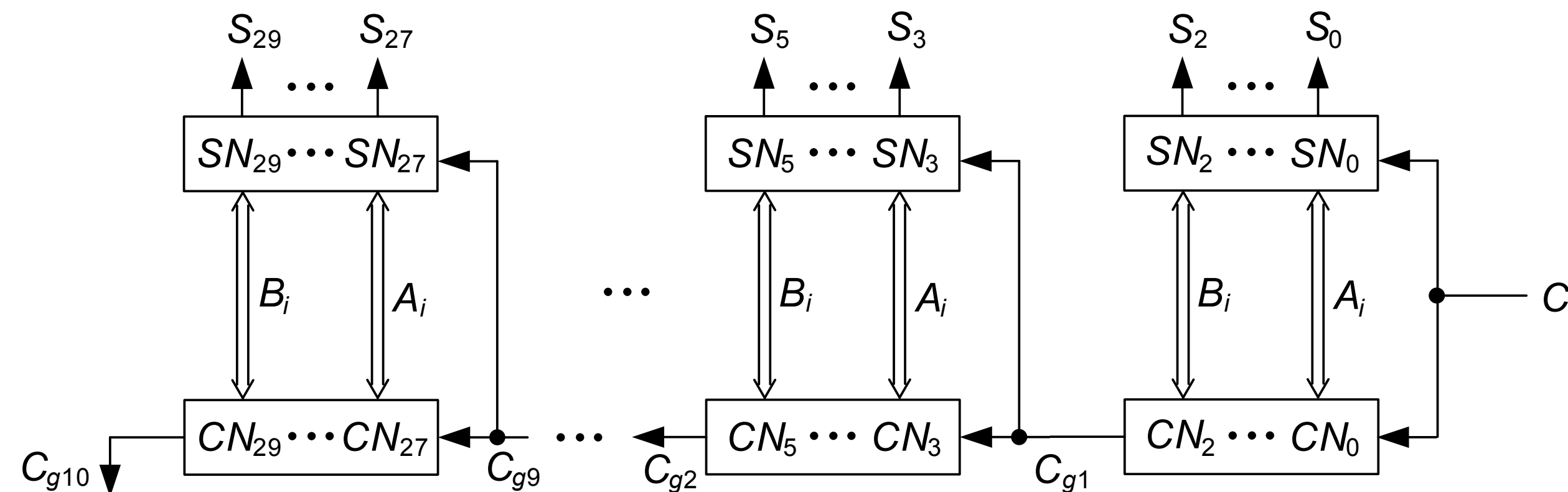
Time taken:

- 4 time units for C_{g1} **why 4?**
- Only 2 time units for C_{g2} and other group carries **why?**

Carry Lookahead Adder

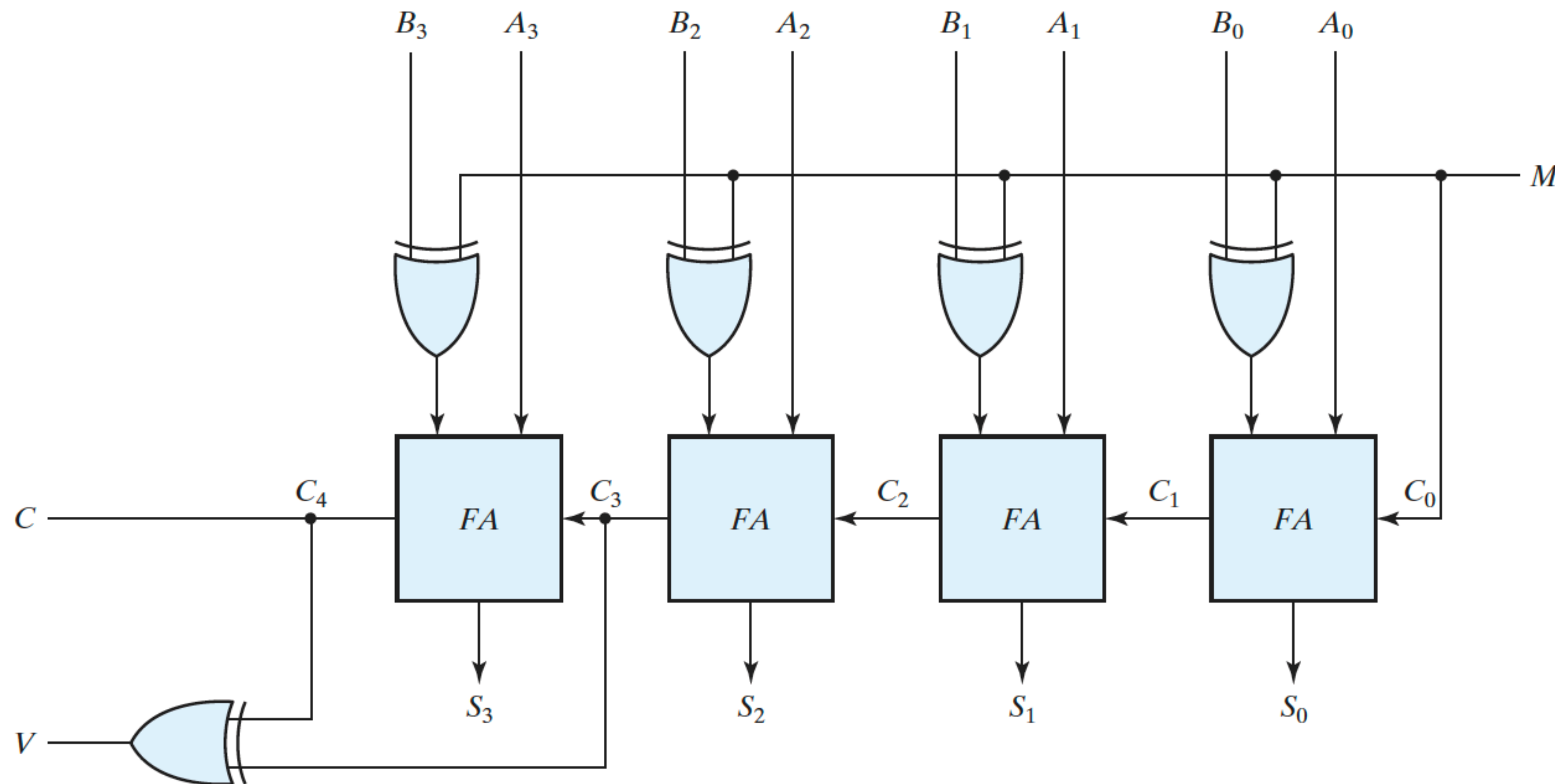
Example: divide n stages into groups of **three bits** for a **30 bit adder**

- **Very very important!!!**
 - First stage requires more time (2 units extra to generate T_i).
 - Last stage requires more time (2 extra units for the final sum, for the other units it's not coming in the critical path)
- **Time taken:** $4 + 2n/3$ time units
- 50% additional hardware for a threefold speedup



Adder Subtractor

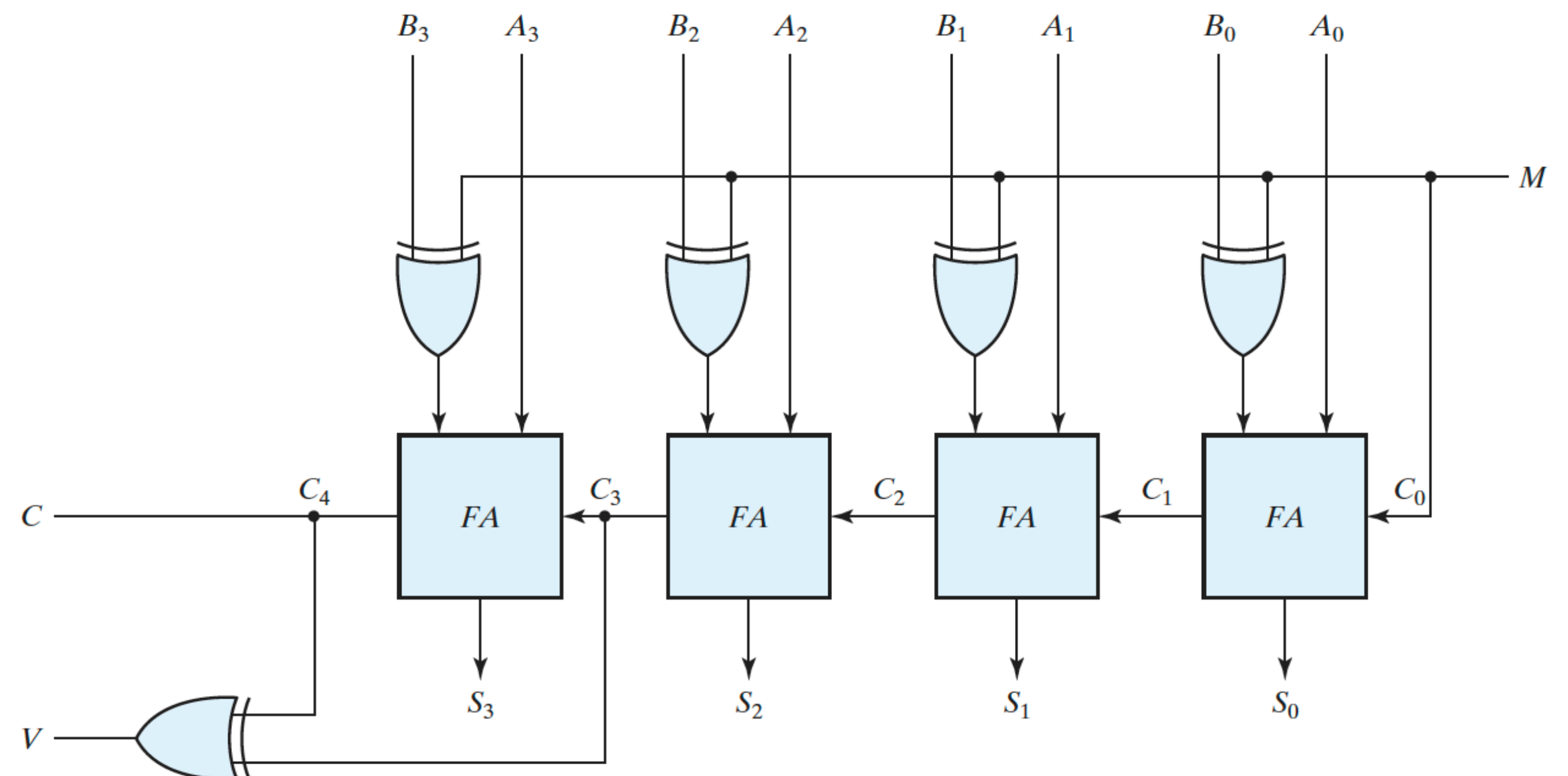
- Implements two's complement subtraction
- The idea is to have both an adder and subtractor in the same circuit
- **Overflow detection**



Adder Subtractor: The Overflow

- **Overflow:** When two numbers with n digits each are added and the sum is a number occupying $n + 1$ digits, we say that an overflow occurred.
- This is true for binary or decimal numbers, signed or unsigned
- An overflow condition can be detected by observing the carry into the sign bit position and the carry out of the sign bit position. If these two carries are not equal, an overflow has occurred.
- If the numbers are unsigned
 - C bit detects the overflow
- If the numbers are signed
 - V bit detects the overflow

carries:	0	1		carries:	1	0
+70	0	1000110		-70	1	0111010
+80	0	1010000		-80	1	0110000
<u>+150</u>	<u>1</u>	<u>0010110</u>		<u>-150</u>	<u>0</u>	<u>1101010</u>



BCD Adder

- Adds two BCD Numbers
- First add in binary and then convert the sum to BCD
- Values beyond 9 requires “correction”
 - For values beyond 9, 0110 needs to be added to generate the BCD encoded sum
- We use a bit C to detect when correction is needed
- $C = 1$, when
 - Carry K of the binary sum is 1
 - $Z_8 = 1$ and $Z_4 = 1$
 - $Z_8 = 1$ and $Z_2 = 1$
- $C = K + Z_8Z_4 + Z_8Z_2$

Binary Sum					BCD Sum					Decimal
K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

BCD Adder

